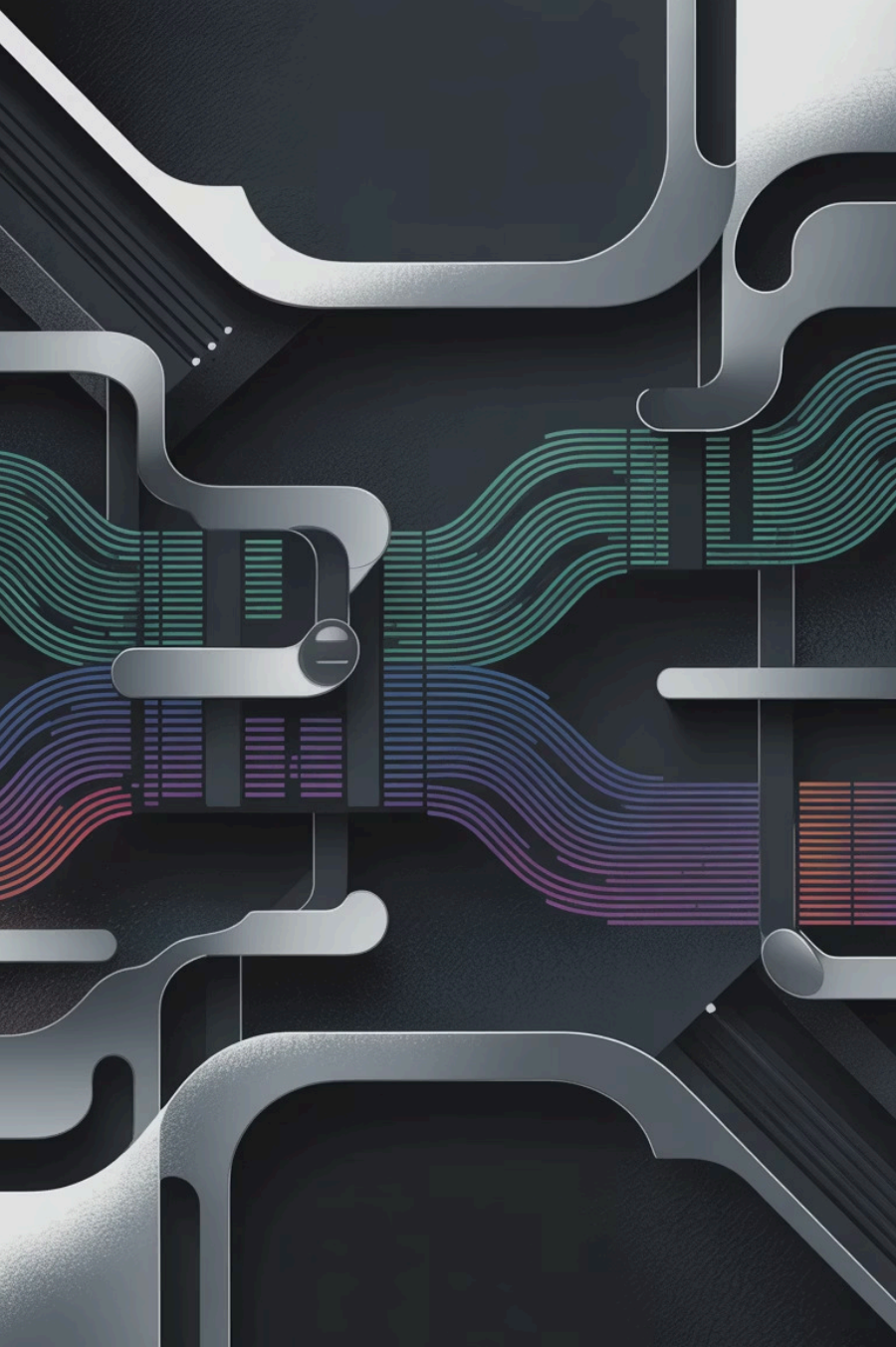




Module 10- Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence



Today's Learning Journey

01

Understanding JSONL Format

Explore the structure and advantages of JSON Lines for LLM data pipelines

02

Memory & Streaming Efficiency

Learn why JSONL enables processing massive datasets without memory overflow

03

Practical Transformations

Convert SQL tables and text corpora into JSONL for training and fine-tuning

04

Hands-On Implementation

Build production-ready ETL pipelines for LLM data preparation

The Data Serialization Challenge

Traditional Formats

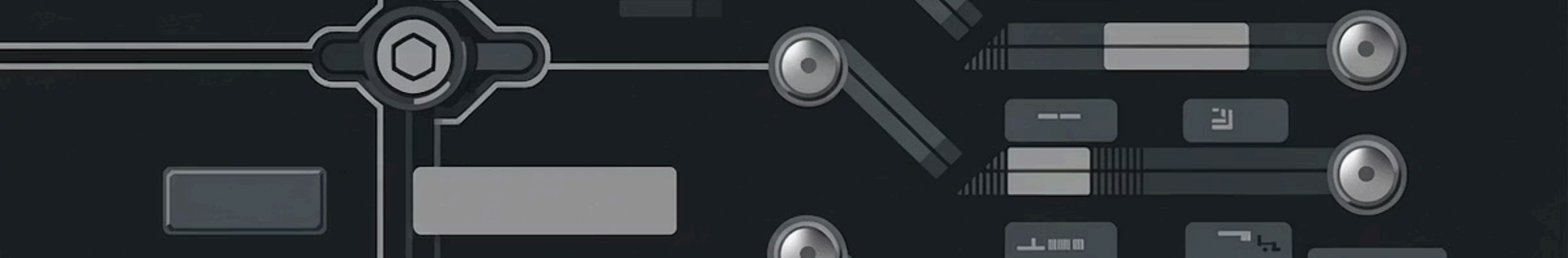
CSV, Excel, and standard JSON files have dominated data storage for decades. However, when working with large language model training data—often containing millions of text samples—these formats create significant bottlenecks.

Loading a 10GB JSON array into memory requires parsing the entire structure before processing begins. This approach becomes impractical when datasets exceed available RAM.

LLM-Scale Requirements

Training and fine-tuning language models demands processing vast text corpora efficiently. A single fine-tuning dataset might contain 100,000+ instruction-response pairs, each with varying lengths.

We need formats that support streaming, parallel processing, and incremental loading while maintaining data integrity and easy parsing.



Enter JSON Lines (JSONL)

JSON Lines, also known as JSONL or newline-delimited JSON, is a text format where each line represents a complete, valid JSON object. This simple structure unlocks powerful capabilities for large-scale data processing.

Line-Delimited Structure

Each line is a self-contained JSON object, terminated by a newline character. No outer array wrapper, no commas between records.

Streaming-Friendly

Process one record at a time without loading the entire file. Perfect for datasets larger than available memory.

Append-Only Updates

Add new records by appending lines to the file. No need to rewrite or parse existing content.

JSONL vs Standard JSON

Standard JSON Array

```
{
  "data": [
    {"text": "First example", "label": "A"},
    {"text": "Second example", "label": "B"},
    {"text": "Third example", "label": "C"}
  ]
}
```

Issues: Must parse entire array, requires closing bracket, memory-intensive for large datasets

JSON Lines Format

```
{"text": "First example", "label": "A"}
{"text": "Second example", "label": "B"}
{"text": "Third example", "label": "C"}
```

Advantages: Stream one line at a time, append new records easily, fault-tolerant processing

Why JSONL for Large Corpora?

Memory Efficiency

Process datasets exceeding available RAM by reading and processing one line at a time. A 50GB training corpus requires only kilobytes of memory during streaming.

- No need to load entire file structure
- Constant memory footprint regardless of file size
- Enables processing on resource-constrained environments

Streaming Processing

Begin processing data immediately without waiting for complete file download or parse. Critical for real-time pipelines and distributed systems.

- Start training while data still downloading
- Process records as they arrive from APIs
- Ideal for online learning scenarios

Parallel Processing

Split files by line ranges and process chunks independently across multiple workers. Each worker can operate on its assigned segment without coordination overhead.

- Distribute across compute clusters
- No synchronization required between workers
- Linear scalability with additional resources



Industry Adoption of JSONL

JSON Lines has become the de facto standard for LLM training data across major AI platforms and frameworks. Its adoption reflects real-world requirements for scalable data engineering.



OpenAI Fine-Tuning

OpenAI fine-tuning requires JSONL where **each line is a chat messages object** (system / user / assistant turns) in Chat Completions format. The older {prompt, completion} format is **deprecated and rejected** for chat models — uploading it fails validation.



Hugging Face Datasets

The datasets library natively supports JSONL for efficient loading and streaming, enabling training on datasets too large for local storage.



Production Pipelines

ETL frameworks like Apache Beam, Spark, and Dask all provide optimized JSONL readers for distributed data processing at scale.

Common LLM Data Structures in JSONL

1

Instruction-Following Format

Used for supervised fine-tuning of instruction-tuned models (e.g., current GPT-4-class and Llama-3.x instruct models).

```
{"instruction": "Summarize this article",  
  "input": "Long article text...",  
  "output": "Brief summary..."}
```

Alpaca-style format for local SFT frameworks (TRL, Axolotl, Unsloth). Hosted APIs like OpenAI use the chat messages format instead (tile 2).

2

Chat Conversation Format

Multi-turn dialogues for training conversational AI, matching the ChatML or messages API structure.

```
{"messages": [  
  {"role": "system", "content": "You are a helpful assistant"},  
  {"role": "user", "content": "What is Python?"},  
  {"role": "assistant", "content": "Python is..."}  
]}
```

3

Text Classification Format

Labeled examples for fine-tuning models on categorization tasks like sentiment analysis or intent detection.

```
{"text": "This product exceeded expectations",  
  "label": "positive",  
  "confidence": 0.95}
```

Memory Efficiency Deep Dive

Understanding memory consumption patterns reveals why JSONL is essential for production LLM pipelines. Let's examine the technical details of how different formats impact system resources.

10GB

JSON Array in Memory

Loading a 10GB JSON file requires parsing the entire structure, typically consuming 15-20GB RAM due to parsing overhead

10MB

JSONL Streaming Buffer

Processing the same 10GB dataset line-by-line requires only a small buffer, typically 10-50MB depending on record size

1000x

Efficiency Gain

The memory reduction enables processing datasets 1000x larger than available RAM through streaming approaches

Streaming vs Batch Loading

Standard JSON (Batch)

```
import json

# Loads entire file into memory
with open('large_dataset.json') as f:
    data = json.load(f)

# Process all records
for record in data['records']:
    process(record)
```

Memory usage: Entire file + Python object overhead

Start latency: Must parse complete file first

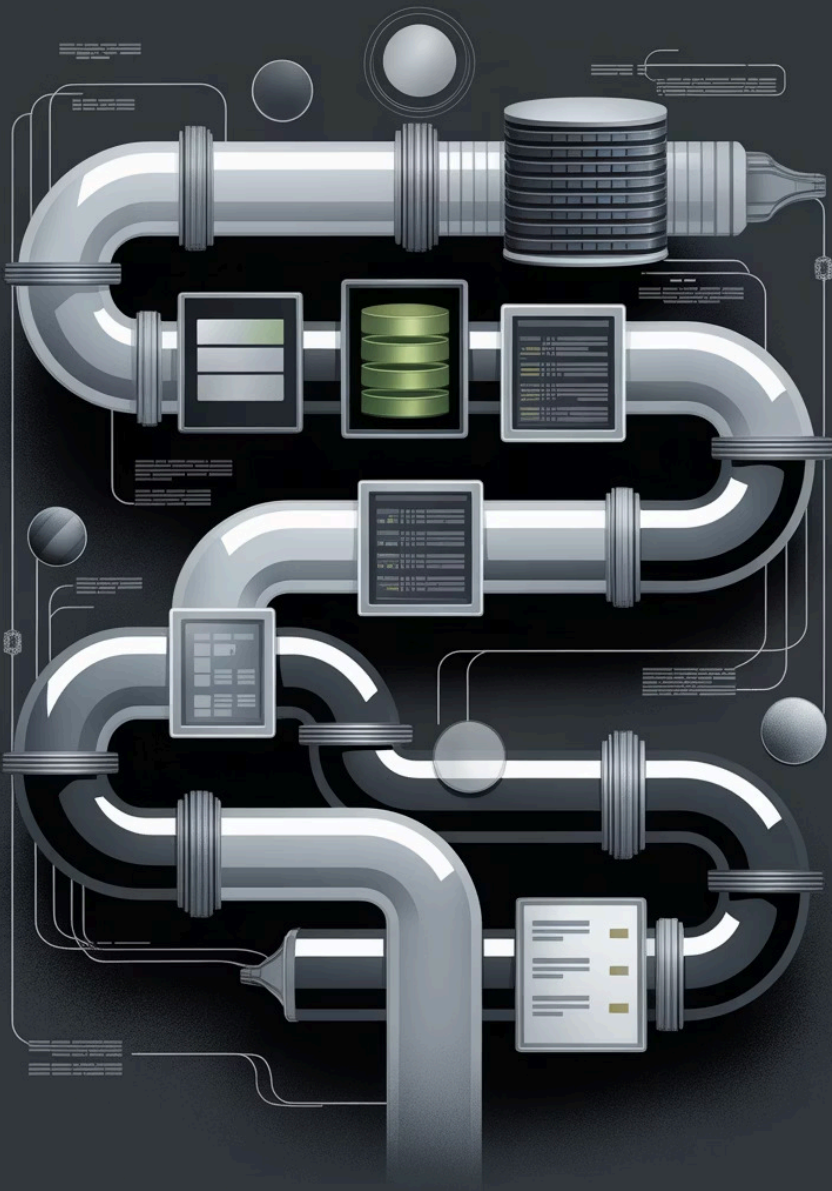
JSONL (Streaming)

```
import json

# Process one line at a time
with open('large_dataset.jsonl') as f:
    for line in f:
        record = json.loads(line)
        process(record)
```

Memory usage: Single record + file buffer only

Start latency: Immediate processing of first line



From SQL Tables to JSONL

Extracting training data from relational databases is a common starting point for LLM fine-tuning. We need to transform normalized table structures into denormalized documents suitable for language models.

1

Extract from SQL

Query databases using Python libraries like `psycopg2`, `pymysql`, or `SQLAlchemy` with cursor-based iteration

2

Transform Schema

Denormalize joins, flatten nested structures, and format text fields into prompt-completion pairs

3

Write JSONL

Stream transformed records directly to `.jsonl` files, writing one complete JSON object per line

Extracting Data with Python: SQL to JSONL

Let's build a production-ready pipeline to extract question-answer pairs from a SQL database and convert them to JSONL format for fine-tuning. This example demonstrates cursor-based streaming to handle large result sets.

```
import psycopg2
import json

# Connect with server-side cursor for streaming
conn = psycopg2.connect(
    host="localhost", database="support_kb",
    user="user", password="pass"
)

# Use server-side cursor to avoid loading all rows
cursor = conn.cursor(name='fetch_qa_pairs')
cursor.execute("""
    SELECT q.question_text, a.answer_text,
           q.category, a.helpful_count
    FROM questions q
    JOIN answers a ON q.id = a.question_id
    WHERE a.is_verified = true
""")

# Stream results directly to JSONL
with open('qa_training_data.jsonl', 'w') as f:
    for row in cursor:
        record = {
            "instruction": "Answer this support question",
            "input": row[0],
            "output": row[1],
            "metadata": {
                "category": row[2],
                "helpfulness": row[3]
            }
        }
        f.write(json.dumps(record) + '\n')

cursor.close()
conn.close()
```

Key Techniques in SQL Extraction



Server-Side Cursors

Named cursors in PostgreSQL enable fetching large result sets in chunks, preventing memory overflow. The database server maintains state while your client iterates.



Batch Fetching

Use `fetchmany(size=1000)` to retrieve records in batches rather than all at once. Balances memory usage with network round-trip overhead.



Data Validation

Filter for quality during extraction: exclude null values, apply length constraints, and verify required fields exist before writing to JSONL.



Immediate Writing

Write each transformed record to the JSONL file immediately. Avoid accumulating records in memory—stream from database to disk.



From Unstructured Text to JSONL

Many LLM training scenarios involve converting raw text corpora—articles, documentation, books—into structured JSONL format. This process requires careful text processing and metadata extraction.

Common Text Sources

- Technical documentation for code generation models
- Customer support transcripts for chatbot training
- Scientific papers for domain-specific models
- Social media conversations for dialogue systems
- Product reviews for sentiment analysis

Transformation Challenges

- Parsing varied document formats (HTML, PDF, Markdown)
- Extracting metadata like authorship, timestamps, categories
- Chunking long documents into model-appropriate segments
- Handling character encoding and special symbols
- Preserving structure while removing formatting artifacts

Processing Text Files to JSONL

This example demonstrates converting a directory of text documents into JSONL format, with each document becoming a training example. We'll add metadata extraction and text cleaning.

```
import json
import os
from pathlib import Path
import re

def clean_text(text):
    """Remove extra whitespace and normalize"""
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

def extract_metadata(filename):
    """Parse filename for metadata"""
    # Example: "tech_docs_python_2023-05-15.txt"
    parts = filename.stem.split('_')
    return {
        "category": parts[1] if len(parts) > 1 else "general",
        "topic": parts[2] if len(parts) > 2 else "unknown",
        "date": parts[3] if len(parts) > 3 else None
    }

# Process all text files in a directory
input_dir = Path("raw_docs")
output_file = "documentation_training.jsonl"

with open(output_file, 'w', encoding='utf-8') as out:
    for filepath in input_dir.glob("*.txt"):
        with open(filepath, 'r', encoding='utf-8') as f:
            content = clean_text(f.read())

        record = {
            "text": content,
            "metadata": extract_metadata(filepath),
            "source": filepath.name,
            "length": len(content)
        }

        out.write(json.dumps(record, ensure_ascii=False) + '\n')
```

Handling Multi-Document Formats

HTML Parsing

Use BeautifulSoup to extract text content, removing scripts, styles, and navigation. Preserve semantic structure from headers and lists.

1

Markdown Processing

Parse with mistletoe or markdown libraries. Preserve code blocks and convert formatting to plain text representations.

3

PDF Extraction

pypdf or pdfplumber to extract text. Handle multi-column layouts, tables, and figure captions appropriately for context.

2

JSON/XML Conversion

Extract relevant fields from structured formats. Flatten nested objects and concatenate text fields as needed.

4



Streaming API Responses to JSONL

Modern data engineering often involves consuming real-time data from APIs. Streaming API responses directly to JSONL files enables building continuously updating training datasets without intermediate storage.

```
import json
import requests

def stream_api_to_jsonl(api_url, output_file, headers=None,
params=None):
    """Stream paginated API responses to JSONL.

    NOTE: pagination keys differ per API. Adjust the two
    marked lines
    for your endpoint's real response shape and page
    parameter.
    """
    params = dict(params or {})
    page = 1
    with open(output_file, 'w', encoding='utf-8') as f:
        while True:
            params["page"] = page
            resp = requests.get(api_url, headers=headers,
                                params=params, timeout=30) # always set
            a timeout
            if resp.status_code != 200:
                break

            items = resp.json() # <-- ADJUST: many APIs
            nest under a key
            if not items: # <-- ADJUST: empty page =
            done
                break

            for item in items:
                f.write(json.dumps(item, ensure_ascii=False) + '\n')
            page += 1

# Example: GitHub's public API paginates with ?page and
# returns a JSON array.
stream_api_to_jsonl(

"https://api.github.com/repos/huggingface/datasets/issues",
"issues.jsonl",
headers={"Accept": "application/vnd.github+json",
        "User-Agent": "training-data-collector"},
params={"per_page": 100, "state": "all"},
)
```

Every API paginates differently (page numbers, cursors, Link headers). This is the *pattern* — adjust the two marked lines to your endpoint's real response shape. Always set a timeout; add retry/backoff for production.

Writing One Record Per Line: Best Practices

Always Add Newline

Every JSONL write must end with `'\n'`. Use `f.write(json.dumps(record) + '\n')` as the standard pattern. Missing newlines break streaming readers.

No Pretty Printing

Never use `indent` parameter in `json.dumps()` for JSONL. Each record must be a single line: `json.dumps(record)` not `json.dumps(record, indent=2)`.

Flush Regularly

Call `f.flush()` periodically for long-running processes. Ensures data is written to disk even if program crashes, preventing data loss.

Handle Encoding

Always specify `encoding='utf-8'` when opening files. Use `ensure_ascii=False` in `json.dumps()` to preserve Unicode characters correctly.

Validate JSON

Test that each record is valid JSON before writing. Catch serialization errors early with try-except blocks around `json.dumps()` calls.

Atomic Writes

For production systems, write to a temporary file first, then rename to final location. Prevents partial files if process terminates unexpectedly.

Chunking Long Documents

Language models have finite context windows. Modern models often accept 128K+ tokens, but you still chunk for retrieval granularity, embedding-model limits, and cost control — so chunking remains a core skill.

Token-Based Chunking

```
from transformers import AutoTokenizer
# NOTE: Model is gated - requires HF auth + license
tokenizer = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-2-7b-hf"
)

def chunk_by_tokens(text, max_tokens=2048,
                    overlap=200):
    """Split text into overlapping chunks"""
    tokens = tokenizer.encode(text)
    chunks = []

    start = 0
    while start < len(tokens):
        end = start + max_tokens
        chunk_tokens = tokens[start:end]
        chunk_text = tokenizer.decode(
            chunk_tokens
        )
        chunks.append(chunk_text)
        start += max_tokens - overlap

    return chunks
```

Writing Chunks to JSONL

```
with open('long_docs.jsonl', 'w') as f:
    for doc in documents:
        chunks = chunk_by_tokens(
            doc['content'],
            max_tokens=2048,
            overlap=200
        )

        for i, chunk in enumerate(chunks):
            record = {
                "text": chunk,
                "doc_id": doc['id'],
                "chunk_index": i,
                "total_chunks": len(chunks),
                "metadata": doc['metadata']
            }
            f.write(json.dumps(record) + '\n')
```




Data Validation During Transformation

Building robust JSONL pipelines requires validating data quality at transformation time. Catching issues early prevents training failures and ensures consistent model behavior.

Schema Validation

Verify each record contains required fields with correct types. Use libraries like `jsonschema` to enforce structure consistently.

- Required field presence
- Data type constraints
- Nested object structure

Content Validation

Check text quality: minimum/maximum lengths, character encoding issues, presence of undesirable content patterns.

- Text length bounds
- Special character filtering
- Language detection

Statistical Profiling

Track distribution statistics during transformation: record counts, field value distributions, null rates, outlier detection.

- Field completeness rates
- Value range validation
- Duplicate detection

Implementing Validation Pipeline

```
import json
from jsonschema import validate, ValidationError

# Define expected schema
SCHEMA = {
    "type": "object",
    "required": ["instruction", "input", "output"],
    "properties": {
        "instruction": {"type": "string", "minLength": 10},
        "input": {"type": "string", "minLength": 1},
        "output": {"type": "string", "minLength": 10, "maxLength": 2000}
    }
}

def validate_record(record):
    """Validate a single training record"""
    try:
        validate(instance=record, schema=SCHEMA)

        # Additional custom validation
        if len(record['output'].split()) < 5:
            return False, "Output too short (word count)"

        if not record['input'].strip():
            return False, "Empty input field"

        return True, None

    except ValidationError as e:
        return False, str(e.message)

# Process with validation
valid_count = 0
error_count = 0

with open('output.jsonl', 'w') as out, \
    open('errors.log', 'w') as err:

    for record in source_records:
        is_valid, error_msg = validate_record(record)

        if is_valid:
            out.write(json.dumps(record) + '\n')
            valid_count += 1
        else:
            err.write(f"Invalid record: {error_msg}\n")
            error_count += 1

print(f"Valid: {valid_count}, Errors: {error_count}")
```

Performance Optimization Techniques



Batch Processing

Process records in batches of 1000-10000 rather than one at a time. Reduces I/O overhead while maintaining memory efficiency.



Multiprocessing

Use Python's multiprocessing to parallelize CPU-intensive transformations. Split input data across workers, merge JSONL outputs.



Buffer Management

Configure appropriate buffer sizes using `buffering` parameter. Larger buffers reduce system calls but increase memory usage.



Compression

For storage efficiency, use gzip compression: `output.jsonl.gz`. Most tools can read compressed JSONL directly via streaming.

Parallel JSONL Generation

For very large datasets, single-threaded processing becomes a bottleneck. Python's multiprocessing library enables parallel transformation with proper JSONL file handling.

```
from multiprocessing import Pool
import json
import os

def process_chunk(chunk_data):
    """Transform a chunk of source data."""
    chunk_id, records = chunk_data
    results = []
    for record in records:
        transformed = transform_record(record)
        is_valid, _ = validate_record(transformed)
        if is_valid:
            results.append(transformed)

    output_file = f"temp_chunk_{chunk_id}.jsonl"
    with open(output_file, 'w', encoding='utf-8') as f:
        for record in results:
            f.write(json.dumps(record) + '\n')
    return output_file

# Split data into chunks
chunks = [(i, source_data[i:i + 10000])
          for i in range(0, len(source_data), 10000)]

# Process in parallel (os.cpu_count() is more portable than a hard-coded 8)
with Pool(processes=os.cpu_count()) as pool:
    chunk_files = pool.map(process_chunk, chunks)

# Merge chunk files into final JSONL
with open('final_output.jsonl', 'w', encoding='utf-8') as final:
    for chunk_file in chunk_files:
        with open(chunk_file, 'r', encoding='utf-8') as chunk:
            final.write(chunk.read())
    os.remove(chunk_file)
```

Reading and Consuming JSONL Files

Once you've generated JSONL training data, you need efficient methods to consume it during model training. Different frameworks provide optimized JSONL readers.

1 Native Python

```
with open('train.jsonl') as f:
    for line in f:
        record = json.loads(line)
        train_model(record)
```

2 Pandas

```
import pandas as pd
df = pd.read_json(
    'train.jsonl',
    lines=True
)
process_dataframe(df)
```

3 Hugging Face

```
from datasets import load_dataset
dataset = load_dataset(
    'json',
    data_files='train.jsonl'
)
trainer.train(dataset)
```


Streaming Large JSONL During Training

For datasets too large to fit in memory, Hugging Face datasets library provides streaming mode that loads data on-demand during training iterations.

```
from datasets import load_dataset

# Enable streaming mode
dataset = load_dataset(
    'json',
    data_files='huge_corpus.jsonl',
    streaming=True,
    split='train'
)

# Shuffle with buffer (not full dataset)
dataset = dataset.shuffle(
    seed=42,
    buffer_size=10000
)

# Iterate without loading all data
for example in dataset:
    tokens = tokenize(example['text'])
    train_step(tokens)
```

Streaming Benefits

- **Constant Memory:** Only buffer in memory at once
- **Immediate Start:** Begin training without waiting for full load
- **Large Datasets:** Train on TB-scale corpora
- **Dynamic Loading:** Fetch from remote storage as needed

The `buffer_size` parameter controls shuffle randomness vs memory usage. Larger buffers provide better shuffling but use more memory.

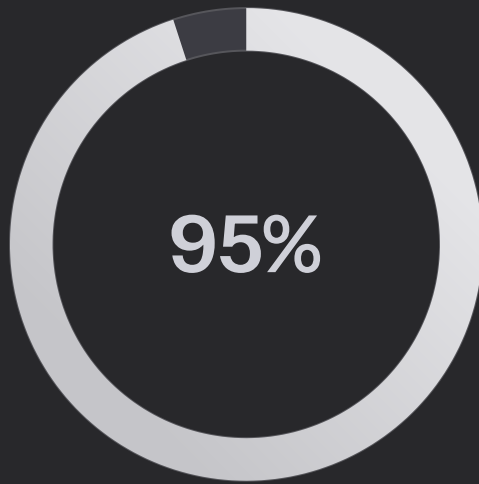
Quality Assurance for JSONL Datasets

Before using JSONL files for training, run comprehensive quality checks to ensure data integrity and identify potential issues that could impact model performance.



Valid JSON Lines

Every line must parse as valid JSON without errors



Schema Compliance

Target threshold for records matching expected structure



Duplicate Rate

Goal for identical records that would waste compute

```
import hashlib

def audit_jsonl(filepath):
    """Comprehensive JSONL quality audit"""
    stats = {
        "total_lines": 0,
        "valid_json": 0,
        "schema_compliant": 0,
        "duplicates": 0,
        "avg_length": 0
    }

    seen_hashes = set()
    lengths = []

    with open(filepath, 'r') as f:
        for line_num, line in enumerate(f, 1):
            stats["total_lines"] += 1

            try:
                record = json.loads(line)
                stats["valid_json"] += 1

                # Check schema
                if validate_record(record)[0]:
                    stats["schema_compliant"] += 1

                # Check duplicates
                record_hash = hashlib.sha256(json.dumps(record, sort_keys=True).encode()).hexdigest()
                if record_hash in seen_hashes:
                    stats["duplicates"] += 1
                seen_hashes.add(record_hash)

                # Track length
                if 'text' in record:
                    lengths.append(len(record['text']))

            except json.JSONDecodeError:
                print(f"Invalid JSON at line {line_num}")

    stats["avg_length"] = sum(lengths) / len(lengths) if lengths else 0
    return stats
```

Common Pitfalls and Solutions

1

Memory Exhaustion

Problem: Accumulating records in list before writing causes out-of-memory errors

Solution: Write each record immediately after transformation, never store full dataset in memory

2

Corrupt Files

Problem: Missing newlines or multi-line JSON breaks streaming readers

Solution: Always use `json.dumps(record) + '\n'` pattern, never pretty-print with indent

3

Encoding Issues

Problem: Unicode characters become garbled or cause crashes

Solution: Explicitly set `encoding='utf-8'` and use `ensure_ascii=False` in `json.dumps()`

4

Silent Failures

Problem: Records fail validation but errors go unnoticed

Solution: Log all validation failures to separate error file, monitor error rates

Production Pipeline Architecture

A robust production system for JSONL generation includes monitoring, error handling, retry logic, and data quality gates. Here's a reference architecture for enterprise-scale LLM data pipelines.

01	02	03
Extract	Transform	Validate
Cursor-based SQL queries or paginated API calls with checkpoint tracking for resume capability	Parallel workers processing data chunks with comprehensive validation and error logging	Schema checks, content validation, duplicate detection, and statistical profiling before load
04	05	
Load	Monitor	
Atomic writes to cloud storage with versioning, checksums, and metadata tracking	Quality metrics, processing rates, error rates, and data drift detection over time	

Key Takeaways



JSONL Enables Scale

Line-delimited JSON is the industry standard for LLM training data because it enables memory-efficient streaming, parallel processing, and incremental updates at any scale.



Stream, Don't Accumulate

Write records immediately after transformation. Never accumulate full datasets in memory—stream from source to destination for production-grade reliability.



Validate Early, Validate Often

Catch data quality issues during transformation with comprehensive schema and content validation. Prevention is easier than debugging failed training runs.



Master the Fundamentals

The patterns you've learned today—cursor-based extraction, streaming writes, parallel processing—are foundational skills for any LLM data engineering role.



Thanks